

Aim: Type a Verilog code to run an Adder and a Counter using IP catalog and upload the code on a FPGA.

Tools and Hardware: Vivado, Arty A7 100t FPGA Board

Theory:

AES128 is a symmetric-key cryptographic algorithm used for secure data encryption and decryption. In this experiment, AES128 encryption and decryption datapaths were implemented on FPGA using Verilog HDL and verified using Xilinx Vivado and Vivado ILA.

Procedure:

1. Open Xilinx Vivado and create a new RTL project for the Arty A7-100T FPGA board
2. Add all AES Verilog source files including aes_system_top, aes_encrypt_round, aes_decrypt_round, subbytes, inv_subbytes, shiftrows, inv_shiftrows, mixcolumns, inv_mixcolumns, sbox, and inv_sbox modules
3. Add S-Box and inverse S-Box memory files (sbox.mem and inv_sbox.mem) to the project
4. Open IP Catalog, search for "ILA", and generate the Integrated Logic Analyzer IP core
5. Instantiate AES encryption, AES decryption, and ILA modules in the top-level Verilog module
6. Connect plaintext, SubBytes, ShiftRows, MixColumns, ciphertext, InvMixColumns, InvShiftRows, InvSubBytes, recovered plaintext, round key, and control signals to ILA probes
7. Add and edit the FPGA constraint (.xdc) file and set aes_system_top.v as the top module

Once the Verilog code is typed and constraint file is edited proceed with the following steps:

1. Run Synthesis
2. Run Synthesis
3. Run Implementation
4. Generate Bitstream
5. Open Hardware Manager
6. Auto-connect the FPGA Board
7. Program the FPGA Board using generated bitstream
8. Observe all AES encryption and decryption stages using Vivado ILA waveforms

AES128 Encryption:

1. AES Encryption Top RTL:

```
1  `timescale 1ns/1ps
2
3  module aes_encrypt_top(
4      input clk,
5      input reset
6  );
7
8      reg [127:0] state;
9      wire [127:0] round_out;
10     wire [127:0] round_key;
11     wire [127:0] subbytes_debug;
12     wire [127:0] shiftrows_debug;
13     wire [127:0] mixcolumns_debug;
14     reg [127:0] plain_text;
15     reg [127:0] key;
16     wire [127:0] cipher_text;
17     wire done;
18
19     initial
20     begin
21         plain_text =
22             128'h48454C4C4F20574F524C440000000000;
23         key =
24             128'h000102030405060708090A0B0C0D0E0F;
25         state =
26             128'h48454C4C4F20574F524C440000000000 ^
27             128'h000102030405060708090A0B0C0D0E0F;
28     end
29
30     assign round_key = key;
31
32     AES_round ROUND(
33
34         .state_in(state),
35         .round_key(round_key),
36
37         .subbytes_debug(subbytes_debug),
38         .shiftrows_debug(shiftrows_debug),
39         .mixcolumns_debug(mixcolumns_debug),
40         .state_out(round_out)
41
42     );
43
44     assign cipher_text = round_out;
45     assign done = 1'b1;
46
47     ila_0 ILA (
48         .clk(clk),
49         .probe0(plain_text),
50         .probe1(subbytes_debug),
51         .probe2(shiftrows_debug),
52         .probe3(mixcolumns_debug),
53         .probe4(cipher_text),
54         .probe5(round_key),
55         .probe6(done)
56     );
57 endmodule
58
```

2. Aes round RTL

```
1 module AES_round(  
2  
3     input [127:0] state_in,  
4     input [127:0] round_key,  
5  
6     output [127:0] subbytes_debug,  
7     output [127:0] shiftrows_debug,  
8     output [127:0] mixcolumns_debug,  
9     output [127:0] state_out  
10  
11 );  
12  
13 wire [127:0] subbytes_out;  
14 wire [127:0] shiftrows_out;  
15 wire [127:0] mixcolumns_out;  
16  
17 subbytes SB(  
18     .data_in(state_in),  
19     .data_out(subbytes_out)  
20 );  
21  
22  
23 shiftrows SR(  
24     .data_in(subbytes_out),  
25     .data_out(shiftrows_out)  
26 );  
27  
28  
29 mixcolumns MC(  
30     .data_in(shiftrows_out),  
31     .data_out(mixcolumns_out)  
32 );  
33  
34  
35  
36  
37  
38 assign state_out = mixcolumns_out ^ round_key;  
39 assign subbytes_debug = subbytes_out;  
40 assign shiftrows_debug = shiftrows_out;  
41 assign mixcolumns_debug = mixcolumns_out;  
42 endmodule
```

3. SubBytes RTL

```
1 module subbytes(  
2     input [127:0] data_in,  
3     output [127:0] data_out  
4 );  
5  
6 genvar i;  
7 generate  
8     for(i = 0; i < 16; i = i + 1)  
9         begin  
10             sbox S(  
11                 .in(data_in[(127-(i*8)) -: 8]),  
12                 .out(data_out[(127-(i*8)) -: 8])  
13             );  
14         end  
15     endgenerate  
16 endmodule
```

4. S-Box RTL

```
1 module sbox(  
2  
3     input [7:0] in,  
4     output [7:0] out  
5  
6 );  
7     reg [7:0] sbox_mem [0:255];  
8     initial  
9     begin  
10         $readmemh("sbox.mem", sbox_mem);  
11     end  
12     assign out = sbox_mem[in];  
13 endmodule
```

5. ShiftRows RTL

```
1 module shiftrows(  
2     input [127:0] data_in,  
3     output [127:0] data_out  
4 );  
5  
6     assign data_out = {  
7         data_in[127:120],  
8         data_in[87:80],  
9         data_in[47:40],  
10        data_in[7:0],  
11  
12        data_in[95:88],  
13        data_in[55:48],  
14        data_in[15:8],  
15        data_in[103:96],  
16  
17        data_in[63:56],  
18        data_in[23:16],  
19        data_in[111:104],  
20        data_in[71:64],  
21  
22        data_in[31:24],  
23        data_in[119:112],  
24        data_in[79:72],  
25        data_in[39:32]  
26    };  
27 endmodule
```

6. MixColumns RTL

```
1 module mixcolumns(  
2  
3     input [127:0] data_in,  
4     output [127:0] data_out  
5  
6 );  
7  
8 function [7:0] xtime;  
9     input [7:0] b;  
10    begin  
11        if(b[7] == 1'b1)  
12            xtime = (b << 1) ^ 8'h1B;  
13        else  
14            xtime = (b << 1);  
15        end  
16    endfunction  
17  
18 function [7:0] mul3;  
19     input [7:0] b;  
20    begin  
21        mul3 = xtime(b) ^ b;  
22    end  
23 endfunction  
24  
25 wire [7:0] s0, s1, s2, s3;  
26 wire [7:0] y0, y1, y2, y3;  
27  
28 assign s0 = data_in[127:120];  
29 assign s1 = data_in[119:112];  
30 assign s2 = data_in[111:104];  
31 assign s3 = data_in[103:96];  
32  
33 assign y0 = xtime(s0) ^ mul3(s1) ^ s2 ^ s3;  
34 assign y1 = s0 ^ xtime(s1) ^ mul3(s2) ^ s3;  
35 assign y2 = s0 ^ s1 ^ xtime(s2) ^ mul3(s3);  
36 assign y3 = mul3(s0) ^ s1 ^ s2 ^ xtime(s3);  
37  
38 wire [7:0] s4, s5, s6, s7;  
39 wire [7:0] y4, y5, y6, y7;  
40  
41 assign s4 = data_in[95:88];  
42 assign s5 = data_in[87:80];  
43 assign s6 = data_in[79:72];  
44  
45 assign s7 = data_in[71:64];  
46  
47 assign y4 = xtime(s4) ^ mul3(s5) ^ s6 ^ s7;  
48 assign y5 = s4 ^ xtime(s5) ^ mul3(s6) ^ s7;  
49 assign y6 = s4 ^ s5 ^ xtime(s6) ^ mul3(s7);  
50 assign y7 = mul3(s4) ^ s5 ^ s6 ^ xtime(s7);  
51  
52 wire [7:0] s8, s9, s10, s11;  
53 wire [7:0] y8, y9, y10, y11;  
54  
55 assign s8 = data_in[63:56];  
56 assign s9 = data_in[55:48];  
57 assign s10 = data_in[47:40];  
58 assign s11 = data_in[39:32];  
59  
60 assign y8 = xtime(s8) ^ mul3(s9) ^ s10 ^ s11;  
61 assign y9 = s8 ^ xtime(s9) ^ mul3(s10) ^ s11;  
62 assign y10 = s8 ^ s9 ^ xtime(s10) ^ mul3(s11);  
63 assign y11 = mul3(s8) ^ s9 ^ s10 ^ xtime(s11);  
64  
65 wire [7:0] s12, s13, s14, s15;  
66 wire [7:0] y12, y13, y14, y15;  
67  
68 assign s12 = data_in[31:24];  
69 assign s13 = data_in[23:16];  
70 assign s14 = data_in[15:8];  
71 assign s15 = data_in[7:0];  
72  
73 assign y12 = xtime(s12) ^ mul3(s13) ^ s14 ^ s15;  
74 assign y13 = s12 ^ xtime(s13) ^ mul3(s14) ^ s15;  
75 assign y14 = s12 ^ s13 ^ xtime(s14) ^ mul3(s15);  
76 assign y15 = mul3(s12) ^ s13 ^ s14 ^ xtime(s15);
```

```

77 assign data_out = {
78
79     y0,
80     y1,
81     y2,
82     y3,
83
84     y4,
85     y5,
86     y6,
87     y7,
88
89     y8,
90     y9,
91     y10,
92     y11,
93
94     y12,
95     y13,
96     y14,
97     y15
98
99 };
100
101 endmodule

```

7. Encryption Testbench

```

1  `timescale 1ns/1ps
2
3  module aes_encrypt_core_tb;
4      reg clock;
5      reg reset;
6      reg start;
7      reg [127:0] plain_text;
8      reg [127:0] key;
9      wire [127:0] cipher_text;
10     wire done;
11
12     aes_encrypt_top DUT(
13         .clock(clock),
14         .reset(reset),
15         .start(start),
16         .plain_text(plain_text),
17         .key(key),
18         .cipher_text(cipher_text),
19         .done(done)
20     );
21
22     initial
23     begin
24         clock = 0;
25         forever #5 clock = ~clock;
26     end
27
28     initial
29     begin
30         reset = 0;          start = 0;
31         plain_text = 128'h48454C4C4F20574F524C440000000000;
32         key = 128'h000102030405060708090A0B0C0D0E0F; #20;
33         reset = 1; #20;
34         start = 1; #10;          start = 0;
35         wait(done);
36         $display("\n=====");
37         $display("PLAINTEXT  = %h", plain_text);
38         $display("KEY       = %h", key);
39         $display("CIPHERTEXT = %h", cipher_text);
40         $display("=====\\n"); #50;
41         $stop;
42     end
43 endmodule

```

Constraint File:

```
1 | #AES_CORE
2 | set_property PACKAGE_PIN E3 [get_ports clk]
3 | set_property IOSTANDARD LVCMOS33 [get_ports clk]
4 | create_clock -add -name sys_clk_pin -period 10.00 [get_ports clk]
5 |
6 | set_property PACKAGE_PIN D9 [get_ports reset]
7 | set_property IOSTANDARD LVCMOS33 [get_ports reset]
```

AES128 Decryption:

1. aes_decrypt_top RTL

```
1 | `timescale 1ns/1ps
2 |
3 | module aes_decrypt_top(
4 |     input clk,
5 |     input reset
6 | );
7 |     reg [127:0] cipher_text;
8 |     reg [127:0] key;
9 |     wire [127:0] round_key;
10 |    wire [127:0] inv_subbytes_debug;
11 |    wire [127:0] inv_shiftrows_debug;
12 |    wire [127:0] inv_mixcolumns_debug;
13 |    wire [127:0] plain_text;
14 |    wire done;
15 |
16 |    initial
17 |    begin
18 |        cipher_text = 128'hBC2AAB09E408011F7021CB8E3C862F8A;
19 |        key = 128'h000102030405060708090A0B0C0D0E0F;
20 |    end
21 |    assign round_key = key;
22 |
23 |    AES_decrypt_round DECRYPT(
24 |        .state_in(cipher_text),
25 |        .round_key(round_key),
26 |        .inv_subbytes_debug(inv_subbytes_debug),
27 |        .inv_shiftrows_debug(inv_shiftrows_debug),
28 |        .inv_mixcolumns_debug(inv_mixcolumns_debug),
29 |        .state_out(plain_text)
30 |    );
31 |    assign done = 1'b1;
32 |    ila_0 ILA (
33 |        .clk(clk),
34 |        .probe0(cipher_text),
35 |        .probe1(inv_subbytes_debug),
36 |        .probe2(inv_shiftrows_debug),
37 |        .probe3(inv_mixcolumns_debug),
38 |        .probe4(plain_text),
39 |        .probe5(round_key),
40 |        .probe6(done)
41 |    );
42 | endmodule
```

2. AES_decrypt_round RTL

```
1  `timescale 1ns/1ps
2
3  module AES_decrypt_round(
4
5      input [127:0] state_in,
6      input [127:0] round_key,
7
8      output [127:0] inv_subbytes_debug,
9      output [127:0] inv_shiftrows_debug,
10     output [127:0] inv_mixcolumns_debug,
11
12     output [127:0] state_out
13 );
14
15
16     wire [127:0] addroundkey_out;
17     wire [127:0] inv_mixcolumns_out;
18     wire [127:0] inv_shiftrows_out;
19     wire [127:0] inv_subbytes_out;
20
21     assign addroundkey_out = state_in ^ round_key;
22
23     InvMixColumns IMC(
24         .data_in(addroundkey_out),
25         .data_out(inv_mixcolumns_out)
26     );
27
28     inv_shiftrows ISR(
29         .data_in(inv_mixcolumns_out),
30         .data_out(inv_shiftrows_out)
31     );
32
33     inv_subbytes ISB(
34         .data_in(inv_shiftrows_out),
35         .data_out(inv_subbytes_out)
36     );
37
38     assign state_out = inv_subbytes_out ^ round_key;
39     assign inv_mixcolumns_debug = inv_mixcolumns_out;
40     assign inv_shiftrows_debug = inv_shiftrows_out;
41     assign inv_subbytes_debug = inv_subbytes_out;
42 endmodule
```

3. inv_mixcolumns RTL

```
1  `timescale 1ns / 1ps
2
3  module InvMixColumns(
4      input [127:0] data_in,
5      output [127:0] data_out
6  );
7
8      function [7:0] xtime;
9          input [7:0] b;
10         begin
11             xtime = (b[7]) ? ((b<<1)^8'h1B) : (b<<1);
12         end
13     endfunction
14
15     function [7:0] mul9;
16         input [7:0] b;
17         begin
18             mul9=xtime(xtime(xtime(b)))^b;
19         end
20     endfunction
21
22     function [7:0] mul11;
23         input [7:0] b;
24         begin
25             mul11=xtime(xtime(xtime(b))) ^ xtime(b) ^ b;
26         end
27     endfunction
28
29     function [7:0] mul13;
30         input [7:0] b;
31         begin
32             mul13=xtime(xtime(xtime(b))) ^ xtime(xtime(b)) ^ b;
33         end
34     endfunction
35
36     function [7:0] mul14;
37         input [7:0] b;
38         begin
39             mul14=xtime(xtime(xtime(b))) ^ xtime(xtime(b)) ^ xtime(b);
40         end
41     endfunction
42
```



```

43     wire [7:0] s [0:15];
44     wire [7:0] o [0:15];
45
46     genvar i;
47     generate
48     for(i=0;i<16;i=i+1)
49         begin: byte_split
50             assign s[i]=data_in[127 -i*8 -: 8];
51         end
52     endgenerate
53
54     assign o[0]=mul14(s[0])^mul11(s[1])^mul13(s[2])^mul9(s[3]);
55     assign o[1]=mul9(s[0])^mul14(s[1])^mul11(s[2])^mul13(s[3]);
56     assign o[2]=mul13(s[0])^mul9(s[1])^mul14(s[2])^mul11(s[3]);
57     assign o[3]=mul11(s[0])^mul13(s[1])^mul9(s[2])^mul14(s[3]);
58
59     assign o[4]=mul14(s[4])^mul11(s[5])^mul13(s[6])^mul9(s[7]);
60     assign o[5]=mul9(s[4])^mul14(s[5])^mul11(s[6])^mul13(s[7]);
61     assign o[6]=mul13(s[4])^mul9(s[5])^mul14(s[6])^mul11(s[7]);
62     assign o[7]=mul11(s[4])^mul13(s[5])^mul9(s[6])^mul14(s[7]);
63
64     assign o[8]=mul14(s[8])^mul11(s[9])^mul13(s[10])^mul9(s[11]);
65     assign o[9]=mul9(s[8])^mul14(s[9])^mul11(s[10])^mul13(s[11]);
66     assign o[10]=mul13(s[8])^mul9(s[9])^mul14(s[10])^mul11(s[11]);
67     assign o[11]=mul11(s[8])^mul13(s[9])^mul9(s[10])^mul14(s[11]);
68
69     assign o[12]=mul14(s[12])^mul11(s[13])^mul13(s[14])^mul9(s[15]);
70     assign o[13]=mul9(s[12])^mul14(s[13])^mul11(s[14])^mul13(s[15]);
71     assign o[14]=mul13(s[12])^mul9(s[13])^mul14(s[14])^mul11(s[15]);
72     assign o[15]=mul11(s[12])^mul13(s[13])^mul9(s[14])^mul14(s[15]);
73 generate
74
75     for(i = 0; i < 16; i = i + 1)
76         begin : byte_join
77             assign data_out[127 - i*8 -: 8] = o[i];
78         end
79     endgenerate
80 endmodule

```

4. inv_shiftrows RTL

```

1 module inv_shiftrows(
2     input  [127:0] data_in,
3     output [127:0] data_out
4 );
5
6     assign data_out = {
7         data_in[127:120],
8         data_in[23:16],
9         data_in[47:40],
10        data_in[71:64],
11
12        data_in[95:88],
13        data_in[119:112],
14        data_in[15:8],
15        data_in[39:32],
16
17        data_in[63:56],
18        data_in[87:80],
19        data_in[111:104],
20        data_in[7:0],
21
22        data_in[31:24],
23        data_in[55:48],
24        data_in[79:72],
25        data_in[103:96]
26    };
27 endmodule

```

5. inv_subbytes RTL

```
1 module inv_subbytes(  
2     input  [127:0] data_in,  
3     output [127:0] data_out  
4 );  
5     genvar i;  
6     generate  
7         for(i = 0; i < 16; i = i + 1)  
8             begin  
9                 inv_sbox S(  
10                    .in(data_in[(127-(i*8)) -: 8]),  
11                    .out(data_out[(127-(i*8)) -: 8])  
12                );  
13            end  
14        endgenerate  
15    endmodule
```

6. inv_sbox RTL

```
1 module inv_sbox(  
2     input  [7:0] in,  
3     output [7:0] out  
4 );  
5  
6     reg [7:0] inv_sbox_mem [0:255];  
7     initial  
8     begin  
9         $readmemh("inv_sbox.mem", inv_sbox_mem);  
10    end  
11    assign out = inv_sbox_mem[in];  
12 endmodule
```

7. Decryption Testbench

```
1 `timescale 1ns/1ps  
2  
3 module aes_decrypt_core_tb;  
4     reg clock;  
5     reg reset;  
6     reg start;  
7     reg [127:0] cipher_text;  
8     reg [127:0] key;  
9     wire [127:0] recovered_plaintext;  
10    wire done;  
11  
12    aes_decrypt_top DUT(  
13        .clock(clock),  
14        .reset(reset),  
15        .start(start),  
16        .cipher_text(cipher_text),  
17        .key(key),  
18        .recovered_plaintext(recovered_plaintext),  
19        .done(done)  
20    );  
21  
22    initial  
23    begin  
24        clock = 0;  
25        forever #5 clock = ~clock;  
26    end  
27    initial  
28    begin  
29        reset = 0;    start = 0;  
30        cipher_text = 128'hBC2AAB09E408011F7021CB8E3C862F8A;  
31        key = 128'h000102030405060708090A0B0C0D0E0F;    #20;  
32        reset = 1;    #20;  
33        start = 1;    #10;  
34        start = 0;    wait(done);  
35        $display("\n=====");  
36        $display("CIPHERTEXT      = %h", cipher_text);  
37        $display("KEY              = %h", key);  
38        $display("RECOVERED PLAINTEXT= %h", recovered_plaintext);  
39        $display("=====\\n");    #50;  
40        $stop;  
41    end  
42 endmodule
```

```
1 | #AES_CORE
2 | set_property PACKAGE_PIN E3 [get_ports clk]
3 | set_property IOSTANDARD LVCMOS33 [get_ports clk]
4 | create_clock -add -name sys_clk_pin -period 10.00 [get_ports clk]
5 |
6 | set_property PACKAGE_PIN D9 [get_ports reset]
7 | set_property IOSTANDARD LVCMOS33 [get_ports reset]
```

Waveform - hw_ila_1

ILA Status: Idle

Name	Value
> plain_text[127:0]	HELLO WORLD
> subbytes__g[127:0]	521b2f84b33fd152b6e2f2bfed7ab76
> shiftrows__g[127:0]	523f2f76b36eab84bd72f52f61bd12b
> mixcolumn__g[127:0]	bc2ba90aa0d0d7187828c185308b2185
> round_key[127:0]	000102030405060708090a0b0c0d0e0f
> cipher_text[127:0]	bc2aab09e408011f7021cb9e3c862f8a
done	1

The waveform display shows a memory dump starting at address 0. The first line of the dump is 'HELLO WORLD'. The subsequent lines show the hexadecimal representation of the string: 521b2f84b33fd152b6e2f2bfed7ab76, 523f2f76b36eab84bd72f52f61bd12b, bc2ba90aa0d0d7187828c185308b2185, 000102030405060708090a0b0c0d0e0f, and bc2aab09e408011f7021cb9e3c862f8a. A red vertical line is positioned at address 400.

The screenshot shows the 'Waveform - hw_ila_1' window. The 'ILA Status' is 'Idle'. The 'Name' column lists variables: 'cipher_text[127:0]', 'inv_subby_g[127:0]', 'inv_shift_ug[127:0]', 'inv_mixco_g[127:0]', 'round_key[127:0]', 'plain_text[127:0]', and 'done'. The 'Value' column shows their current values: 'bc2aab0...', '48444e...', '521b2f84...', '523f2f76...', '0001020...', 'HELLO WORLD', and '1'. The main area displays a memory dump with addresses from 0 to 1,000. A red vertical line is at address 500. The dump shows hex data for 'cipher_text' and 'inv_subby_g', and ASCII data for 'plain_text' ('HELLO WORLD'). The status bar at the bottom indicates 'Updated at: 2026-May-20 16:06:20'.

The AES128 Encryption and Decryption System was successfully implemented and verified on FPGA using Verilog HDL, Xilinx Vivado, and Vivado ILA. All AES encryption and decryption stages were successfully analyzed through RTL design, simulation, and real-time hardware verification.